

Flutter Transmit Security Authentication - User Guide

Overview

The Flutter Transmit Security Authentication module provides secure authentication methods including PIN codes, biometrics, and WebAuthn for Flutter applications. This guide covers installation, setup, and usage.

Note: For additional integration guides such as configuring passkeys and advanced platform-specific features, we recommend reading the native SDK documentation:

- [iOS documentation \(https://developer.transmitsecurity.com/guides/webauthn/quick_start_sdk_ios/\)](https://developer.transmitsecurity.com/guides/webauthn/quick_start_sdk_ios/)
- [Android documentation \(https://developer.transmitsecurity.com/guides/webauthn/quick_start_sdk_android/#Command-line\)](https://developer.transmitsecurity.com/guides/webauthn/quick_start_sdk_android/#Command-line)

Installation

1. Add to pubspec.yaml

```
dependencies:  
  flutter_ts_authentication:  
    git:  
      url: https://github.com/TransmitSecurity/flutter_ts_authentication.git  
      ref: v0.0.2 # Use the latest version tag
```

2. Install dependencies

```
flutter pub get
```

Platform Setup

iOS Setup

1. Minimum Requirements

- iOS 15.0+
- Xcode 14+

2. Update ios/Runner/Info.plist

Add biometric permission:

```
<key>NSFaceIDUsageDescription</key>  
<string>This app uses Face ID for secure authentication</string>
```

Note: This module uses Swift Package Manager (SPM) for iOS dependencies. No additional CocoaPods setup is required.

Android Setup

1. Minimum Requirements

- Android API 23+
- compileSdkVersion 34+

2. Add Maven repository to android/build.gradle

```
allprojects {  
    repositories {  
        google()  
        mavenCentral()  
        maven {  
            url "https://transmit.jfrog.io/artifactory/transmit-security-gradle-release-local/"  
        }  
    }  
}
```

3. Update android/app/build.gradle

```
android {  
    compileSdkVersion 34  
  
    defaultConfig {  
        minSdkVersion 23  
        targetSdkVersion 34  
    }  
}
```

4. Add permissions to android/app/src/main/AndroidManifest.xml

```
<uses-permission android:name="android.permission.USE_BIOMETRIC" />  
<uses-permission android:name="android.permission.USE_FINGERPRINT" />  
<uses-permission android:name="android.permission.INTERNET" />
```

5. (Optional) Configure biometric prompt strings in android/app/src/main/res/values/strings.xml

```
<string name="BiometricPromptTitle">Authenticate with Biometrics</string>  
<string name="BiometricPromptSubtitle">Use your device biometrics to authenticate.</string>  
<string name="BiometricPromptCancel">Cancel</string>
```

These strings customize the biometric authentication dialog shown to users.

Integration

Important Setup Requirements

Maven Repository Configuration

Add the Maven repository in **both** the module-level and app-level build.gradle files:

Module-level android/build.gradle:

```
allprojects {
    repositories {
        google()
        mavenCentral()
        maven {
            url "https://transmit.jfrog.io/artifactory/transmit-security-gradle-release-local/"
        }
    }
}
```

App-level android/app/build.gradle:

```
repositories {
    google()
    mavenCentral()
    maven {
        url "https://transmit.jfrog.io/artifactory/transmit-security-gradle-release-local/"
    }
}
```

Android MainActivity Configuration

IMPORTANT: Make sure that your MainActivity inherits from FlutterFragmentActivity:

```
import io.flutter.embedding.android.FlutterFragmentActivity

class MainActivity: FlutterFragmentActivity() {
    // Your activity code
}
```

Why FlutterFragmentActivity?

- Required for biometric authentication features
- Provides proper fragment management for native biometric dialogs
- Ensures compatibility with Android's biometric prompt APIs

Basic Usage

1. Import the module

```
import 'package:flutter_ts_authentication/flutter_ts_authentication.dart';
```

2. Initialize the SDK

```

final auth = FlutterTsAuthentication();

// Initialize SDK
await auth.initializeSDK();

// Initialize with configuration
final initOptions = TSInitOptions(
  webAuthnInitOptions: TSWebAuthnInitOptions(
    startAuthentication: '/auth/webauthn/authenticate',
    startRegistration: '/auth/webauthn/register',
  ),
);

await auth.initialize(
  'your-client-id',
  'your-domain.com',
  'https://api.your-domain.com',
  initOptions
);

```

Initialization Parameters

- **clientId**: Your Transmit Security client ID
- **domain**: Your application domain
- **baseUrl**: API endpoint URL (default: <https://api.transmitsecurity.io>)
- **initOptions**: Configuration options containing WebAuthn settings
 - **webAuthnInitOptions**: WebAuthn-specific configuration
 - **rpId**: Relying Party ID (usually your domain)
 - **rpName**: Human-readable name for your application

API Reference

PIN Code Authentication

Register PIN Code

```

try {
  final result = await auth.registerPinCode('username', '123456');
  print('Public Key ID: ${result.publicKeyId}');
  print('Context ID: ${result.contextIdentifier}');

  // Commit the registration
  await auth.commitPinRegistration(result.contextIdentifier);
} catch (e) {
  print('PIN registration failed: $e');
}

```

Authenticate with PIN Code

```

try {
  final result = await auth.authenticatePinCode(
    'username',
    '123456',
    'challenge-string'
  );
  print('Authenticated! Public Key ID: ${result.publicKeyId}');
  print('Signature: ${result.signature}');
} catch (e) {
  print('PIN authentication failed: $e');
}

```

Biometric Authentication

Register Biometrics

```

try {
  final result = await auth.registerNativeBiometrics('username');
  print('Biometric registered: ${result.publicKeyId}');
} catch (e) {
  print('Biometric registration failed: $e');
}

```

Unregister Biometrics

```

try {
  final result = await auth.unregisterNativeBiometrics('userId');
  print('Biometric unregistered: ${result.publicKeyId}');
} catch (e) {
  print('Biometric unregistration failed: $e');
}

```

Authenticate with Biometrics

```

try {
  final result = await auth.authenticateNativeBiometrics(
    'username',
    'challenge-string'
  );
  print('Biometric auth successful: ${result.signature}');
} catch (e) {
  print('Biometric authentication failed: $e');
}

```

Approval with Biometrics

```
try {
  final result = await auth.approvalNativeBiometrics(
    'username',
    'challenge-string'
  );
  print('Approval successful: ${result.signature}');
} catch (e) {
  print('Biometric approval failed: $e');
}
```

WebAuthn

Check WebAuthn Support

```
final isSupported = await auth.isWebAuthnSupported();
if (isSupported) {
  // Proceed with WebAuthn operations
} else {
  // Use alternative authentication method
}
```

Register WebAuthn

```
try {
  final result = await auth.registerWebAuthn('username', 'Display Name');
  print('WebAuthn registered: ${result.result}');
} catch (e) {
  print('WebAuthn registration failed: $e');
}
```

Authenticate with WebAuthn

```
try {
  final result = await auth.authenticateWebAuthn('username');
  print('WebAuthn auth successful: ${result.result}');
} catch (e) {
  print('WebAuthn authentication failed: $e');
}
```

WebAuthn Transaction Signing

```
try {
  final result = await auth.signWebauthnTransaction('username');
  print('Transaction signed: ${result.result}');
} catch (e) {
  print('Transaction signing failed: $e');
}
```

WebAuthn Approval

```
// Method 1: With approval data
final approvalData = {'transaction': 'transfer', 'amount': '100'};
final options = <String>['option1', 'option2'];

try {
  final result = await auth.approvalWebAuthn(
    approvalData,
    'username',
    options
  );
  print('Approval successful: ${result.result}');
} catch (e) {
  print('WebAuthn approval failed: $e');
}

// Method 2: With raw authentication data
final authData = TSWebAuthnAuthenticationData(data: {
  'webauthnSessionId': 'session-id',
  'credentialRequestOptions': {}
});

try {
  final result = await auth.approvalWebAuthnWithData(authData, options);
  print('Approval successful: ${result.result}');
} catch (e) {
  print('WebAuthn approval failed: $e');
}
```

Device Information

```
try {
  final deviceInfo = await auth.getDeviceInfo();
  print('Device Public Key ID: ${deviceInfo.publicKeyId}');
  print('Device Public Key: ${deviceInfo.publicKey}');
} catch (e) {
  print('Failed to get device info: $e');
}
```

Logging Configuration

Enable/Disable Logging

```

try {
  // Enable detailed logging for debugging
  final success = await auth.setLoggingEnabled(true);
  if (success) {
    print('Logging enabled successfully');
  }

  // Disable logging for production
  final disabled = await auth.setLoggingEnabled(false);
  if (disabled) {
    print('Logging disabled successfully');
  }
} catch (e) {
  print('Failed to configure logging: $e');
}

```

Note: Logging configuration helps with debugging authentication issues during development. It's recommended to disable logging in production builds for security and performance reasons.

TOTP (Time-based One-Time Password)

Register TOTP

```

try {
  final result = await auth.registerTOTP(
    'otpauth://totp/Example:user@example.com?secret=JBSWY3DPEHPK3PXP&issuer=Example',
    TSTOTPSecurityType.biometric // or TSTOTPSecurityType.none
  );

  print('TOTP registered successfully');
  print('Issuer: ${result.issuer}');
  print('Label: ${result.label}');
  print('UUID: ${result.uuid}');
} catch (e) {
  print('TOTP registration failed: $e');
}

```

Security Types:

- TSTOTPSecurityType.biometric: Requires biometric authentication to generate codes
- TSTOTPSecurityType.none: No additional security required

Generate TOTP Code

```

try {
  final result = await auth.generateTOTPCode('your-totp-uuid');
  print('Generated TOTP code: ${result.code}');
} catch (e) {
  print('Failed to generate TOTP code: $e');
}

```

Generate TOTP Code with Challenge

```
try {
  final result = await auth.generateTOTPCodeWithChallenge(
    'your-totp-uuid',
    'challenge-string'
  );
  print('Generated TOTP code with challenge: ${result.code}');
} catch (e) {
  print('Failed to generate TOTP code with challenge: $e');
}
```

Use Cases:

- **Basic TOTP:** Use generateTOTPCode() for standard time-based authentication
- **Challenge-based TOTP:** Use generateTOTPCodeWithChallenge() for transaction signing or enhanced security scenarios

Device Key Signing

Sign challenges using device keys for secure authentication and transaction validation:

```
try {
  final TSSignChallengeResult result = await auth.signWithDeviceKey('challenge-to-sign');
  print('Signature: ${result.signature}');
} catch (e) {
  print('Device key signing failed: $e');
}
```

Use Cases:

- **Transaction Signing:** Sign transaction data for secure approval workflows
- **Challenge-Response:** Implement challenge-response authentication protocols
- **Device Authentication:** Prove device identity using cryptographic signatures
- **Secure Communications:** Sign messages for end-to-end security verification

Error Handling

All methods throw exceptions on failure. Wrap calls in try-catch blocks:

```

try {
    final result = await auth.authenticatePinCode('user', '1234', 'challenge');
    // Success
} on PlatformException catch (e) {
    switch (e.code) {
        case 'INVALID_ARGUMENTS':
            print('Invalid parameters provided');
            break;
        case 'NULL_RESULT':
            print('No result returned from native platform');
            break;
        default:
            print('Authentication failed: ${e.message}');
    }
} catch (e) {
    print('Unexpected error: $e');
}

```

Data Types

TSPinCodeRegistrationCompletion

```

class TSPinCodeRegistrationCompletion {
    final String publicKeyId;
    final String publicKey;
    final String keyType;
    final String contextIdentifier;
}

```

TSPinCodeAuthenticationCompletion

```

class TSPinCodeAuthenticationCompletion {
    final String publicKeyId;
    final String signature;
    final String challenge;
}

```

TSBiometricsRegistrationResult

```

class TSBiometricsRegistrationResult {
    final String publicKey;
    final String publicKeyId;
    final String os;
    final String keyType;
    final String? attestation; // Biometric attestation data (optional)
}

```

TSBiometricsAuthenticationResult

```
class TSTBiometricsAuthenticationResult {  
    final String publicKeyId;  
    final String signature;  
}
```

TSNativeBiometricsUnregisterResult

```
class TSNativeBiometricsUnregisterResult {  
    final String publicKeyId; // Public key ID of the unregistered biometric  
}
```

DeviceInfo

```
class DeviceInfo {  
    final String publicKeyId;  
    final String publicKey;  
}
```

TSTOTPSecurityType

```
enum TSTOTPSecurityType {  
    biometric, // Requires biometric authentication to generate codes  
    none      // No additional security required  
}
```

TSTOTPRegistrationCompletion

```
class TSTOTPRegistrationCompletion {  
    final String? issuer; // TOTP issuer (optional)  
    final String? label;  // TOTP label (optional)  
    final String uuid;    // Unique identifier for the TOTP  
}
```

TSTOTPGenerateCodeCompletion

```
class TSTOTPGenerateCodeCompletion {  
    final String code; // Generated TOTP code  
}
```

Troubleshooting

Common Issues

1. iOS Build Errors

- Ensure iOS deployment target is 15.0+
- Module uses Swift Package Manager (SPM) - no CocoaPods setup needed

2. Android Build Errors

- Check minSdkVersion is 23+
- Verify all required permissions are added

3. R8/ProGuard Issues (Release Builds)

- The plugin includes consumer ProGuard rules automatically
- If you encounter minification issues, add these rules to your app's android/app/proguard-rules.pro:

```
# Keep TSAAuthentication SDK classes
-keep class com.transmit.authentication.** { *; }
-keep interface com.transmit.authentication.** { *; }
```

4. Biometric Not Working

- Check device has biometric hardware
- Verify permissions are granted
- Ensure user has enrolled biometrics

5. WebAuthn Not Supported

- Use `isWebAuthnSupported()` to check availability
- WebAuthn requires specific platform support

6. TOTP Issues

- Ensure TOTP URI format is correct: `otpauth://totp/...`
- Verify the UUID from registration is stored and used correctly
- For biometric security type, ensure biometrics are set up on device
- Check that TOTP registration completed successfully before generating codes

Debug Tips

- Enable verbose logging in debug builds
- Test on real devices for biometric features
- Check platform-specific error messages
- Verify network connectivity for WebAuthn

Support

For issues and questions:

- Check the troubleshooting section
- Review platform-specific requirements
- Test on multiple devices and OS versions